RoSteALS Implementation and Robustness

Eugene Francisco

June 2026

Overview

This writeup describes my implementation of Bui et al.'s[1] RoSteALS technique for image watermarking. I include some deeper training details and some findings during training which the original paper did not discuss.

As an overview of the technique, RoSteALS embeds watermarks into a cover image by using an image autoencoder and embedding information within the cover image's latent representation. More formally, let E, G be an autoencoder. For a high fidelity image $c \in \mathbb{R}^{H\times W\times C}$, let $E(c) = z \in \mathbb{R}^{H'\times W'\times C}$ be c's latent representation. For a latent variable $z \in\in \mathbb{R}^{H'\times W'\times C}$, let $G(z) \in \mathbb{R}^{H\times W\times C}$ be the generated image from the auto encoder.

How does RoSteALS watermark images? Let $c \in \mathbb{R}^{H\times W\times C}$ be an image we intend to watermark and $z = E(c)$ be its encoding. Let $s \in \mathbb{R}^{\ell}$ be a secret message we intend to embed in c. RoSteALS learns two neural networks:

(i) First, it learns a message encoder F_θ which outputs $\delta = F_\theta(s) \in \mathbb{R}^{H'\times W'\times C}$. The watermarked latent is then $z+\delta$ and the watermarked image is $\tilde{c} = G(z + \delta)$.

(ii) Second, it leans a secret decoder D_φ which outputs $\tilde{s} = D_\varphi(\tilde{c})$, a reconstruction of the original message.

The neural networks are parametrized by learnable parameters θ and φ. Note that the autoencoder E, G is not learned and is fixed in advance. They are trained using two objectives:

(a) $\mathcal{L}_{\text{quality}} = \mathcal{L}_{\text{LPIPS}}(\tilde{c}, c) + \alpha\|\gamma(\tilde{c}) - \gamma c\|_2^2$, where $\mathcal{L}_{\text{LPIPS}}$ is the LPIPS image reconstruction loss and γ is the mapping function from RGB to YUV.

(b) $\mathcal{L}_{\text{recovery}} = \text{BCE}(s, \tilde{s})$ where BCE is the binary cross entropy between s and $\tilde{s}$.

The training loss is then

$$\mathcal{L} = \beta\mathcal{L}_{\text{quality}} + \mathcal{L}_{\text{recovery}}.$$

[1] https://arxiv.org/pdf/2304.03400

Implementation Details

The purple text corresponds to implementation details from Bui et al. The olive text corresponds to my own specific implementation details. The VQ-GAN vq-f4 autoencoder is used; this uses a latent dimension of $H' = H/4$ and $W' = W/4$. I fixed hyperparameters $\alpha = 1.5$ and β is scaled dynamically (see below). I use the AdamW optimizer with learning rate $2 \cdot 10^{-5}$. I trained on the COCO training dataset from 2017 with $100{,}000$ images of size 256×256 and a message length of $\ell = 50$. I used curriculum learning to train with the following training schedule:

(a) *Warmup phase 1*: I fix $\beta = 0.1$ for the entirety of the phase and train F_θ and D_φ until the bit accuracy reaches 0.9. Training is done with a fixed subset of 8 randomly selected images from the total training set and a minibatch size of 4.

(b) *Warmup phase 2:* I reveal an additional $50{,}000$ images. β continues to be fixed at 0.1. Batch size is bumped to 8. This phase ends once bit accuracy reaches 0.8.[2]

(c) *Exposure phase 1:* Over the course of $5{,}000$[3] training iterations, I linearly increase β from 0.1 to 10.

(d) *Exposure phase 2:* I reveal the full $100{,}000$ image training set to the models. This phase ends once bit accuracy reaches 0.98.

(e) *Noising phase:* I insert a noising function N between the creation of each watermarked image $\tilde{c}$ and message decoding. This means that the message decoder outputs $D(N(\tilde{c}))$.[4]

For message encoder, I use the following Neural Network

[2] In my experiments, the transition from *warmup phase 1* to *warmup phase 2* is typically followed by a drastic drop in bit accuracy, hence the lowered threshold.

[3] From my experiments, $5{,}000$ corresponded roughly to a 10 percentage point increase in bit accuracy

[4] Check Bui et al. for specifics on the noising functions used.

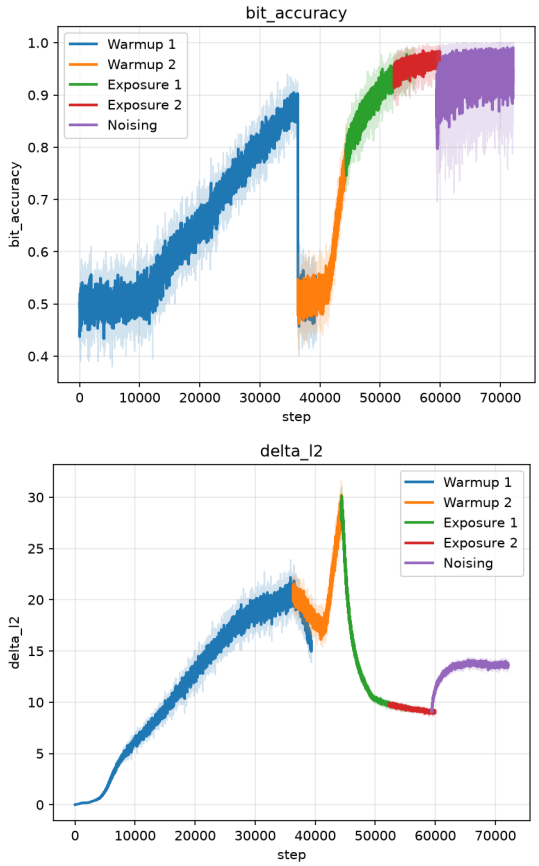

Figure 1: *top:* message recovery bit accuracy; *bottom:* ℓ_2 norm of the latent watermark δ.

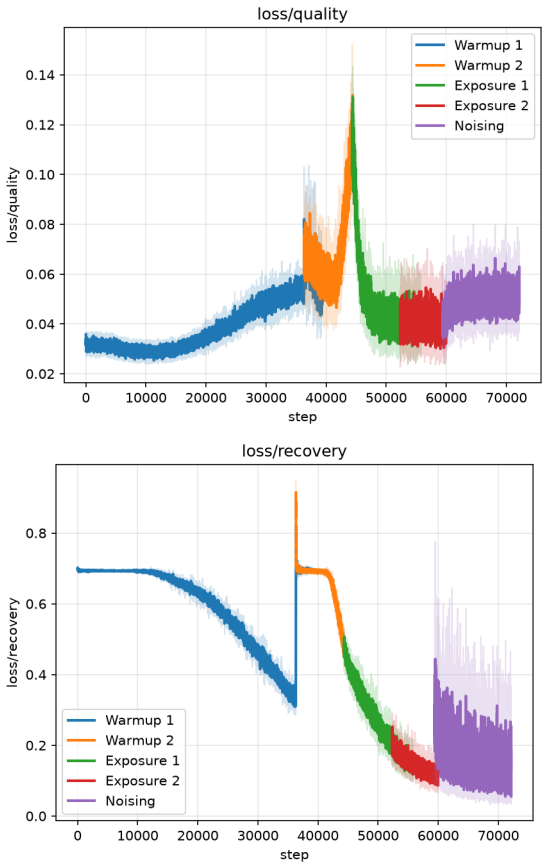

Figure 2: *top:* $\mathcal{L}_{\text{quality}}$; *bottom:* $\mathcal{L}_{\text{recovery}}$.

```
nn.Linear(50, 32*32*3), nn.SiLU,
nn.Unflatten(1, (3, 32, 32)),
nn.Upsample(scale_factor=2),
nn.Conv2d(3, 3, 3, padding=1),
nn.Conv2d(3, 3, 1)
```

where the last convolution layer is 0 initialized. The secret decoder is implemented as a ResNet-50 with the standard ImageNet head replaced by an MLP to $\mathbb{R}^{50}$ of logits representing the decoded message. I loaded ImageNet weights into the ResNet for training. Training was done between an NVIDIA A10 and A100 GPU, with checkpoint (a) being trained with an A10 and the rest with A100s. Training took around 10 hours.

Discussion:

The curriculum learning schedule described here exposes the model to only two mini-batches of data until bit-accuracy reaches 0.9. This leads to significant overfitting and a sudden plummet in bit accuracy once a larger training set is revealed; however, my experiments suggest that this early training phase is critical.

In a separate experiment, I tried the same curriculum learning routine but revealing the model to $15{,}000$ training examples at the start instead of only 8. In that trial, bit accuracy stagnated at around $15{,}000$ for the entire training run, revealing that overfitting on these minibatches is critical for training and perhaps allows the model to learn an encoding scheme to then apply to future images. This is possibly supported by Figure 1 which shows how $\|\delta\|_2$ stays large when we begin exposing the model to more training examples. Because of this, it is possible that the choice of initial mini-batch can affect model robustness and quality which could allow for future work.

Loading the model with ImageNet weights was also critical to training; I tried a couple of training runs with randomly initialized Resnet50 weights to

no avail.